Smooth isn't fast. Smooth is on time.

How I made a WPF app play 4K60 video with live face blur at 60 FPS, up from 7.



Before: 7 FPS **After: 60 FPS**

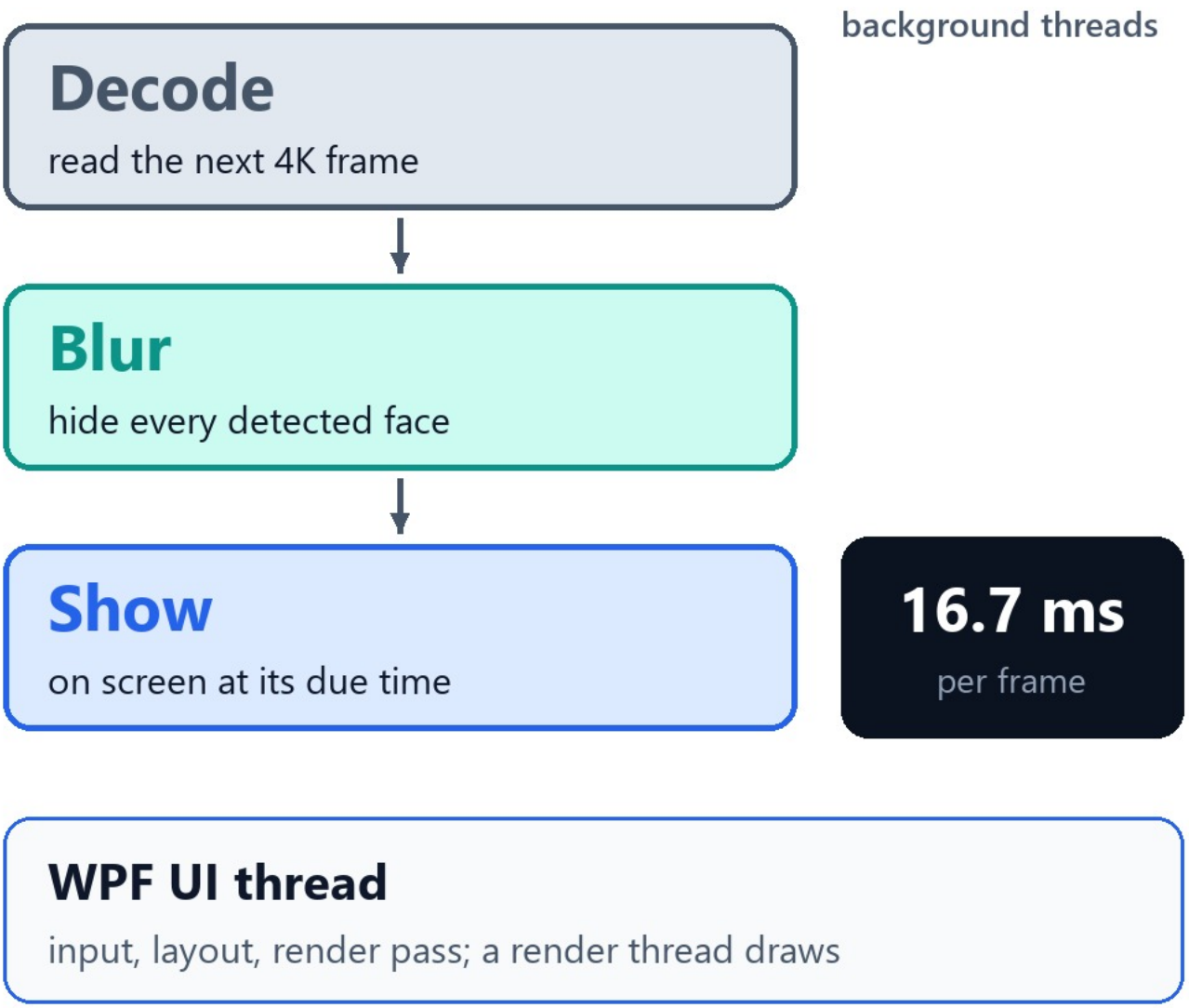
C# · .NET 10 · WPF · OpenCV · FFmpeg

The fast YOLO model (5 scans a second) misses some small, distant faces: that is what the larger model and the review step are for.

Samit Desai · linkedin.com/in/samit-desai-62868420

Three jobs, one deadline

At 60 FPS, every 16.7 ms a new 4K frame must be decoded, blurred and shown.

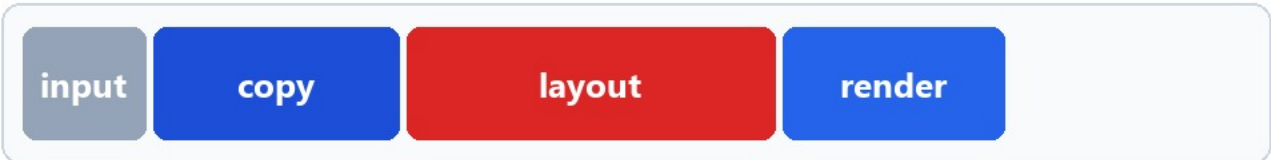


WPF runs input, layout and the render pass on one UI thread

The time readout and the face-box overlay changed on every frame, and each change asked WPF to measure and arrange again: one layout pass per frame, 60 a second at 60 FPS.

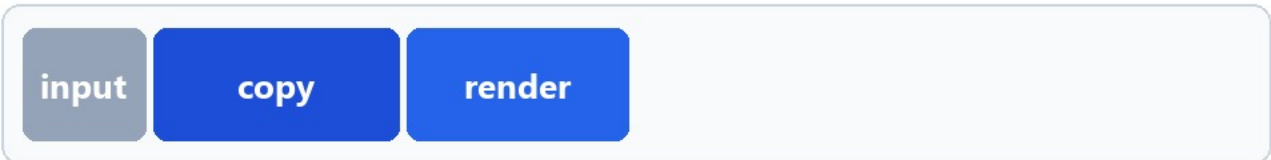
illustration, not to scale

Before: one 16.7 ms frame



60 layout passes / s

After: one 16.7 ms frame



0 layout passes / s

copy	WritePixels copies the new video frame into the bitmap (about 0.6 ms at 1080p, 2.5–2.8 ms at 4K)
layout	measure + arrange: every element's size and position (≈ 1 ms a frame before, 0 after, measured)
render	the render pass hands what changed, the new frame included, to WPF's render thread, which draws it on the GPU

Pixels change. Layout doesn't.

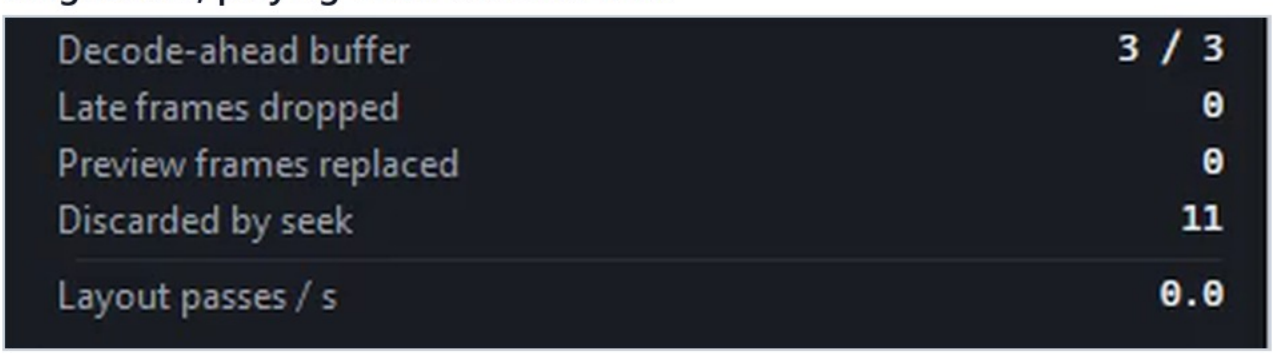
60 → **0**

layout passes per second while playing (1080p60, measured)

62 ms → 0 ms of UI-thread time per second (≈ 1 ms of every frame)

- Video: WritePixels into one reused WriteableBitmap. Same object, new pixels: no binding, no layout.
- Face boxes: re-recorded into a retained drawing. The playhead: moved by a transform.
- Changing text: RenderOnlyText re-records a retained drawing instead of a TextBlock.

Diagnostics, playing 4K60 with live blur:



Layout passes 0.0 · Presented 60.2 FPS · 0 late frames

Every frame has a due time. Memory stays flat.

Paced by a clock



A frame more than one interval late is dropped. The UI thread gets frames through a one-slot mailbox: a newer frame replaces a waiting one, so it is never more than one frame behind.

Bounded buffer



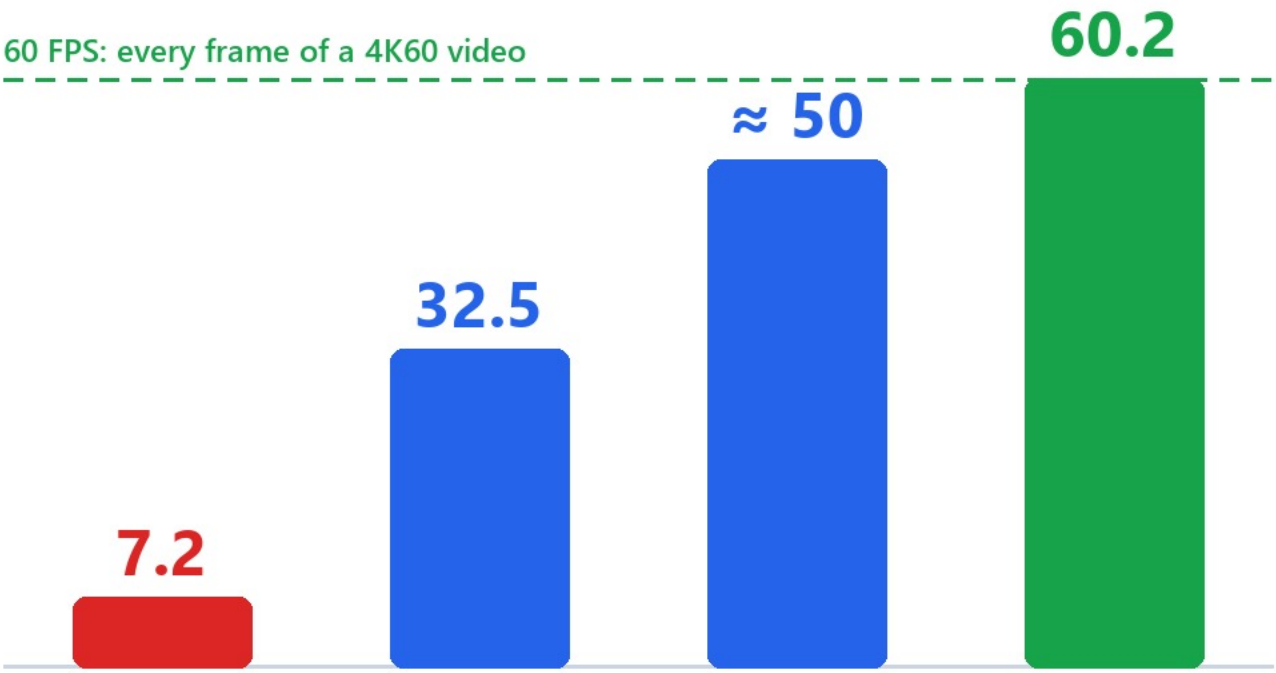
Memory doesn't grow with the video's length: playback holds at most 9 frames by design, and no measured run ever had more than 10 alive.

Diagnostics, 4K60 with live blur:

buffer 3 / 3 · 6 live frames · 0 late frames

The 4K60 stress test: 7 → 60 FPS

4K60 video with live blur, measured in the app on my laptop.



Before
One thread decoded and blurred; late frames were dropped only at the screen: slow motion.

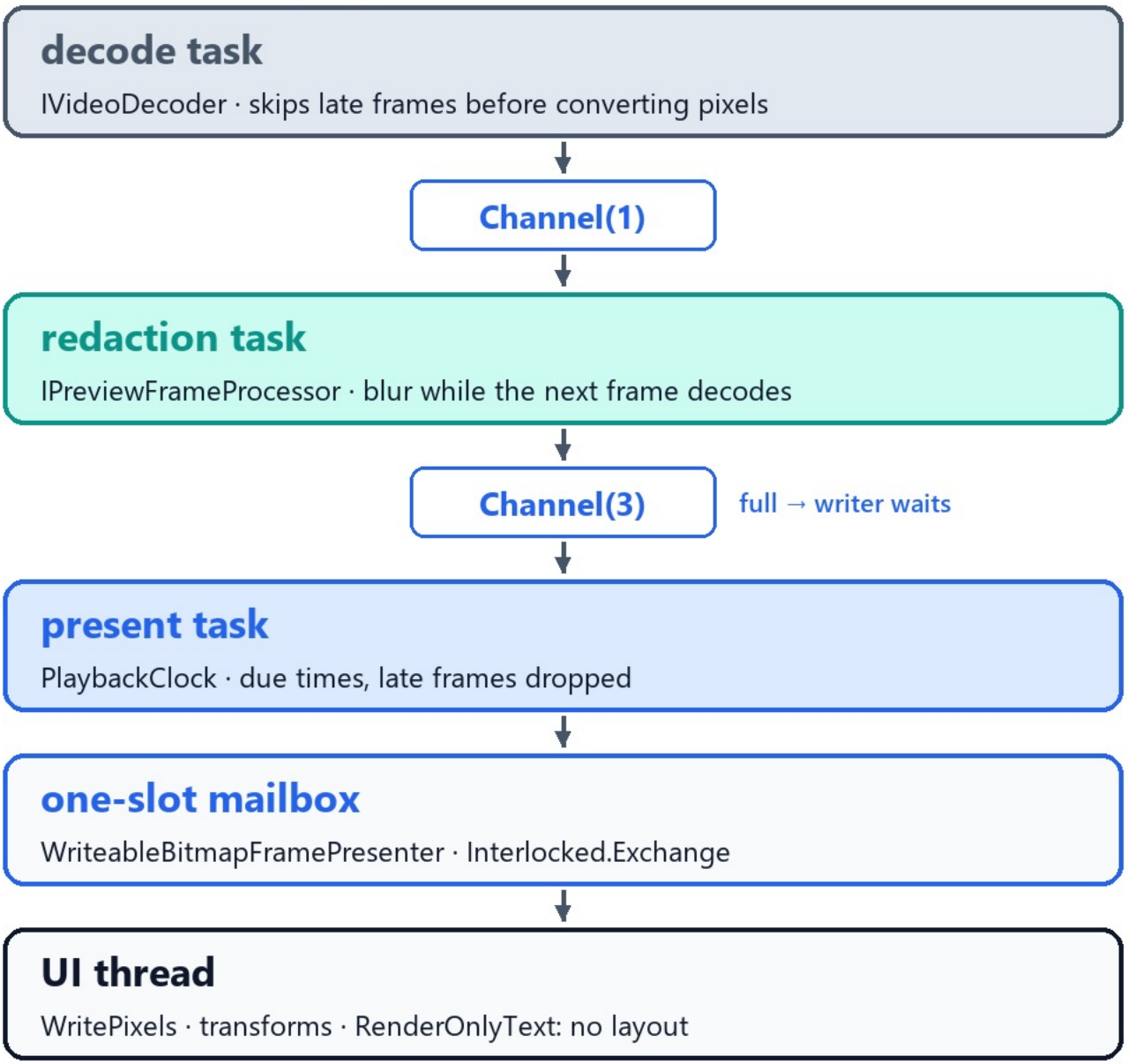
Skip before the work
A frame that can't make it is skipped before its pixels are converted.

Decode || blur
Two threads: the next frame decodes while this one is blurred (47–52).

Thread pool 22 → 4
0 late frames; 14.4 → 5.9 CPU cores.

All four steps measured in the app. Steps 1–3 ran with the PC in use; step 3 measured again on an idle PC gave 50.3–51.9 FPS, so step 4 (idle) compares like with like.

The real pipeline



Seeks bump a generation number, so frames decoded before a seek are thrown away.

Trade-offs, and what's next

- Preview may skip a late frame. Export never drops one.
- Two threads cost more CPU than one.
- Frames are paced by a timer: about 7–8 % of frame intervals run long, not seen as drops.

On a smaller PC: 4 performance cores

simulated by CPU affinity on this laptop

1080p30	**30.0 FPS**	every frame
1080p60	**59.9 FPS**	every frame
4K30	**30.0 FPS**	every frame
4K60	**31.8 FPS**	skips frames to keep up

Next episode

YouTube-style timeline previews in WPF.

Where it's going: the whole pipeline on the GPU.

I built it with an AI coding assistant as my pair programmer; the design, the measurements and the mistakes are mine.

Samit Desai · linkedin.com/in/samit-desai-62868420